

LUMRA ERP

Page Architecture Blueprint

BAGIAN 1: MASTER DATA & SETUP

Disusun: 05 May 2026

DAFTAR ISI

1. PENDAHULUAN
2. HALAMAN KONFIGURASI & SETUP
 - Settings / Konfigurasi Umum
 - Setup Lokasi (Locations)
 - Setup Kategori Produk
 - Setup Unit Pengukuran
3. MASTER DATA - INVENTORY
 - Daftar Produk
 - Tambah/Edit Produk
 - Stok Awal / Adjustment
4. MASTER DATA - PENJUALAN
 - Daftar Customer
 - Tambah/Edit Customer
 - Daftar Vendor
 - Tambah/Edit Vendor
5. MASTER DATA - PRODUKSI
 - Daftar Recipe
 - Tambah/Edit Recipe
6. LAMPIRAN

1. PENDAHULUAN

1.1 Struktur Master Data

Master Data adalah fondasi sistem ERP. Data ini bersifat relatif statis namun sangat penting karena menjadi referensi untuk semua transaksi. Master Data LUMRA mencakup: Lokasi, Kategori, Unit, Produk, Customer, Vendor, dan Recipe.

1.2 Prinsip Desain Page Architecture

- 1. Form Progresif: Tampilkan field esensial terlebih dahulu
- 2. Validasi Real-time: Cek keunikan kode saat user mengetik
- 3. Master-Detail Pattern: Daftar di halaman utama, form terpisah
- 4. Bulk Operations: Sediakan opsi import CSV
- 5. Audit Trail: Simpan siapa yang membuat/edit dan kapan
- 6. Soft Delete: Gunakan flag `is_active` bukan delete fisik

2. HALAMAN KONFIGURASI & SETUP

2.1 Settings / Konfigurasi Umum

Nama Halaman & File Path	settings/general_settings.html
Konsep UI/UX	Tabs: Business Info, Fiscal Year, Currency. Form: Nama perusahaan, Alamat, NPWP, Logo
Logika View (Backend)	GET: Load settings dari LumraConfigBusiness. POST: Validate & update. Decorators: @login_required, @permissi
Koneksi Database	Model: LumraConfigBusiness. Read: company_name, address, currency, fiscal_year_start. Update: semua field
Alur Data	User -> Settings Page -> Submit Form -> Update DB -> Success Message -> Dashboard

2.2 Setup Lokasi (Locations)

Nama Halaman & File Path	settings/locations_list.html & location_form.html
Konsep UI/UX	List tabel: Kode, Nama, Tipe, Status. Actions: Edit, Delete (soft), Detail. Form modal untuk add/edit
Logika View	GET (list): pagination & filtering. GET (form): pre-fill. POST: validate code unique. DELETE: soft delete
Koneksi Database	Model: LumraConfigLocations. Read: id, code, name, location_type. Update: name, address, location_type
Alur Data	List -> Add -> Modal Form -> Submit -> Validate -> Save -> List Refresh

2.3 Setup Kategori Produk

File Path: settings/categories_list.html

UI/UX: Tree view hierarchical dengan expand/collapse

Backend: GET tree structure dengan parent-child. POST validate circular reference

Database: ProductCategories dengan FK self-referencing (parent_id)

Flow: Tree View -> Add/Edit -> Modal -> Validation -> Save -> Tree Refresh

2.4 Setup Unit Pengukuran

File Path: settings/units_list.html

UI/UX: Simple list: Code, Name, Abbreviation. Action buttons

Backend: Prevent delete jika sudah digunakan di products

Database: Units model. Check FK di ProductItems sebelum delete

Flow: List -> Add -> Form -> Validate Unique -> Save -> List

3. MASTER DATA - INVENTORY

3.1 Daftar Produk (Product List)

File Path: inventory/products_list.html

UI/UX: Tabel: SKU, Nama, Kategori, Unit, Status, Stok. Filter kategori & status. Search SKU/nama. Bulk actions. Pagination 25 items/page

Backend: GET dengan pagination, filtering, search. Prefetch Category & Unit. Hitung stok real-time dari InventoryTransaction. Cache 5 menit

Database: ProductItems (main), ProductCategories (FK), Units (FK). Aggregate stok dari InventoryTransaction

Flow: Dashboard -> Inventory -> Products -> List. User: View Detail, Edit, Duplicate, Delete (soft)

3.2 Tambah/Edit Produk (Product Form)

File Path: inventory/product_form.html

UI/UX: Tabs: Basic Info (Code, Name, Category, Unit, Description), Pricing (Buy/Sell price, Markup %), Stock (Initial stock, Min level, Reorder qty), Attributes

Backend: GET (add) form kosong + list kategori & unit. GET (edit) pre-fill. POST validate code unique. Insert/update ProductItems. Jika stok awal > 0 insert InventoryTransaction

Database: ProductItems, ProductCategories, Units. Write InventoryTransaction jika needed

Flow: Product List -> Add -> Form (Tabs) -> Fill Data -> Submit -> Validate -> Save -> Success -> List

3.3 Stok Awal / Adjustment

File Path: inventory/stock_adjustment.html

UI/UX: Select Lokasi. Tabel item: Product, Current Qty, New Qty, Difference. Add row button. Reason field (Opname, Rusak, Hilang). Preview total

Backend: GET form dengan list lokasi & current stok. POST validate qty > 0. Create InventoryTransaction per item. Create audit log

Database: InventoryTransaction (write), LumraConfigLocations (read)

Flow: Inventory -> Stock Adjustment -> Select Location -> Adjust Qty -> Submit -> Audit Log -> Success

4. MASTER DATA - PENJUALAN

4.1 Daftar Customer (Customer List)

File Path: sales/customers_list.html

UI/UX: Tabel: Kode, Nama, Email, Phone, City, Status. Filter status & city. Search nama/email. Action: View Profile, Edit, Delete, Deactivate. Metrics: Last Purchase Date, Total Value

Backend: GET customers + aggregate last purchase & total dari Orders. Prefetch LoyaltyStampCards

Database: LumraConfigCustomers (main), LoyaltyStampCards (FK). Aggregate: Last purchase & Total dari Orders

Flow: Sales -> Customers -> List -> Click Customer -> Profile / Edit / Purchase History

4.2 Tambah/Edit Customer (Customer Form)

File Path: sales/customer_form.html

UI/UX: Tabs: Personal Info (Code, Nama, Email, Phone, DOB), Address, Contact, Settings (Credit limit, Payment term, Loyalty tier)

Backend: GET (add) form + list loyalty tiers. GET (edit) pre-fill. POST validate email & code unique

Database: LumraConfigCustomers. Write: name, email, phone, address, city, credit_limit

Flow: Customer List -> Add -> Form (Tabs) -> Fill -> Submit -> Validate Unique -> Save -> List

4.3 Daftar Vendor (Vendor List)

File Path: purchasing/vendors_list.html

UI/UX: Tabel: Kode, Nama, Email, Phone, City, Status. Filter status & supplier category. Search. Metrics: Total Orders, Last Order Date, Payment Term

Backend: GET vendors + aggregate total orders & last order dari PurchaseOrders

Database: LumraConfigVendors (main), PurchaseOrders (FK)

Flow: Purchasing -> Vendors -> List -> Click -> Profile / Edit / Orders History

4.4 Tambah/Edit Vendor (Vendor Form)

File Path: purchasing/vendor_form.html

UI/UX: Tabs: Company Info (Code, Name, Tax ID, Website), Address, Contact, Supplier Categories (Checkboxes), Payment Terms

Backend: GET (add) form + list supplier categories. GET (edit) pre-fill + selected categories. POST validate code & tax_id unique. Save M2M categories

Database: LumraConfigVendors, VendorCategories (M2M)

Flow: Vendor List -> Add -> Form (Tabs) -> Fill -> Submit -> Validate -> Save -> List

5. MASTER DATA - PRODUKSI

5.1 Daftar Recipe (Recipe List)

File Path: production/recipes_list.html

UI/UX: Tabel: Kode, Nama, Output Product, Yield Qty, Status. Filter category & status. Search recipe name. Action: View, Edit, Duplicate, Archive. Tooltip: Bahan utama

Backend: GET recipes + prefetch output product. Show ingredient count

Database: Recipes (main), ProductItems (FK for output)

Flow: Production -> Recipes -> List -> Click -> View / Edit / Duplicate

5.2 Tambah/Edit Recipe (Recipe Form)

File Path: production/recipe_form.html

UI/UX: Tabs: Basic Info (Code, Name, Output Product, Yield Qty, Unit), Ingredients (Tabel dengan Product, Qty, Unit, Cost; Add row; Auto-calculate total), Instructions (WYSIWYG), Attributes

Backend: GET (add) form + product dropdown. GET (edit) pre-fill + ingredient rows. POST validate output product exists, yield qty > 0. Save Recipes + RecipeIngredients

Database: Recipes, RecipeIngredients (FK to Recipes & ProductItems)

Flow: Recipe List -> Add -> Form (Tabs) -> Add Ingredients -> Submit -> Validate -> Save

6. LAMPIRAN

6.1 Standard View Function Contract

- LIST VIEW: GET dengan pagination, filtering, searching, prefetch relations
- FORM VIEW: GET (render form kosong/prefill), POST (validate & save)
- DELETE VIEW: Soft delete (set is_active=False), bukan delete fisik
- DECORATORS: @login_required, @permission_required sesuai keperluan
- ERROR HANDLING: Try-catch untuk database errors, return meaningful message
- AUDIT LOG: Log siapa, apa, kapan untuk setiap create/update/delete

6.2 Form Validation Checklist

- ✓ Unique constraint: Code, Email, Tax ID (check DB sebelum save)
- ✓ Required fields: Semua field wajib terisi (form validation)
- ✓ Format validation: Email (regex), Phone (11-12 digits), Date (ISO)
- ✓ Business logic: Code pattern (PREFIX-NNNN), Credit limit > 0
- ✓ Foreign key: Parent exists sebelum save child
- ✓ Circular reference: Jika hierarchical, cek tidak ada self-reference
- ✓ Soft delete: Set is_active=False, jangan delete fisik
- ✓ Audit log: Simpan who, what, when untuk perubahan

LUMRA ERP

Page Architecture Blueprint

BAGIAN 2: SALES, POS & INVENTORY ADVANCED

Disusun: 05 May 2026

DAFTAR ISI - BAGIAN 2

1. POS SYSTEM (POINT OF SALE)

- Dashboard POS
- Halaman Transaksi POS
- Cart Management
- Payment & Receipt

2. SALES ORDER & INVOICE

- Daftar Sales Order
- Form Sales Order
- Daftar Invoice
- Detail Invoice

3. INVENTORY ADVANCED

- Stock Opname Workflow
- Approval Workflow
- Stock Transfer Between Locations

4. STOCK OPNAME WORKFLOW

- Select Location
- Input Hitungan
- Approval List
- Session History

5. LOYALTY & CUSTOMER ENGAGEMENT

- Loyalty Dashboard
- Stamp Card Management
- Redemption History

6. LAMPIRAN & BEST PRACTICES

1. POS SYSTEM (POINT OF SALE)

1.1 Dashboard POS

File Path: sales/pos_dashboard.html

UI/UX: Quick stats cards: Total Sales Today, Transaction Count, Top Products, Active Sessions. Button besar untuk 'Start New Transaction' & 'View Session'. Multi-location selector. Filter by date range

Backend: GET: Aggregate transactions dari Orders dengan date filter. Calculate metrics real-time dari cache atau DB

Database: LumraConfigOrders (main), OrderLines (for details). Aggregate: COUNT, SUM(total_amount)

Flow: POS Menu -> Dashboard -> Click 'Start New Transaction' -> Go to POS Terminal

Architecture: Cache metrics 10 menit. Use Q objects untuk multi-location filtering

1.2 Halaman Transaksi POS (POS Terminal)

File Path: sales/pos_terminal.html

UI/UX: Split screen: LEFT (Product grid/list dengan search & categories), RIGHT (Cart dengan items, qty, price, subtotal, tax, total). Buttons: Add to Cart (auto), Void Item, Discount, Pay, Cancel Transaction. Customer field. Payment method selector. Real-time calculation

Backend: GET: Load products, categories, tax settings, customer list. AJAX endpoint untuk: Add Item to Cart, Update Qty, Calculate Tax

Database: ProductItems (read: price, available qty), ProductCategories (read: list), TaxSettings (read: tax_rate)

Frontend Tech: Alpine.js atau Vue untuk cart state management. Real-time validation: Check product exists, qty <= available stock

Flow: Search Product -> Click Add -> Auto calculate total -> Select Customer (optional) -> Pay -> Create Order

Special Handling: Cart stored in session/localStorage. Auto-save every 5s. Prevent submit jika ada validation error

1.3 Cart Management (AJAX Endpoint)

File Path: sales/api/pos_cart.py

Endpoint: POST /api/pos/cart/add, /api/pos/cart/update, /api/pos/cart/remove

Logic:

- ADD: POST product_id, qty -> Check stock -> Add to session cart -> Return updated cart JSON
- UPDATE: POST item_id, qty -> Validate qty <= available -> Update session -> Return updated cart
- REMOVE: POST item_id -> Remove dari session -> Return updated cart
- DISCOUNT: POST discount_type (percentage/fixed), amount -> Calculate & apply -> Return total

Response: {success, cart_items[], subtotal, tax, discount, total, message}

Database: ProductItems (check available qty). Query: select_for_update untuk prevent race condition

1.4 Payment & Receipt

File Path: sales/pos_payment.html

UI/UX: Modal atau separate page. Display: Order summary, payment method selector (Cash, Debit, Credit), amount input, change calculation. Print receipt button. Email receipt option

Backend: POST: Create LumraConfigOrders record, OrderLines (per item), InventoryTransaction (update stok), LoyaltyStampTransactions (earn stamps). If payment method = credit -> create AR (Accounts Receivable) record

Database: LumraConfigOrders (write), OrderLines (write), InventoryTransaction (write), LoyaltyStampTransactions (write)

Special Features: Auto-print receipt (WeasyPrint). Email receipt if customer email provided. Warranty tracking

Flow: Cart -> Click Pay -> Select Payment Method -> Enter Amount -> Create Order -> Print Receipt -> Update Loyalty -> Success

Error Handling: Stock suddenly sold out -> Show error & reduce cart qty automatically

2. SALES ORDER & INVOICE

2.1 Daftar Sales Order

File Path: sales/sales_orders_list.html

UI/UX: Tabel: SO No, Customer, Order Date, Total Amount, Status (Draft, Confirmed, Shipped, Delivered, Cancelled). Filter: Status, Date Range. Search: SO No atau Customer. Action: View, Edit, Convert to Invoice, Cancel

Backend: GET: Fetch SalesOrders + prefetch customer, order lines. Filter & search. Pagination

Database: SalesOrders (main), OrderLines (FK), LumraConfigCustomers (FK)

Flow: Sales -> Sales Orders -> List -> Click SO -> View Details / Convert to Invoice / Edit (if Draft) / Cancel

2.2 Form Sales Order (Create/Edit)

File Path: sales/sales_order_form.html

UI/UX: Tabs: Header (SO No, Customer, Order Date, Delivery Date, Terms), Items (Line item tabel dengan Product, Qty, Unit Price, Discount, Total), Summary (Subtotal, Tax, Freight, Grand Total), Notes

Backend: GET (create) form kosong + customer dropdown. GET (edit) pre-fill SO data + line items. POST: Validate customer exists, at least 1 line item, qty > 0. Create SalesOrders + SalesOrderLines. Prevent edit jika status != Draft

Database: SalesOrders (write), SalesOrderLines (write), LumraConfigCustomers (read for dropdown)

Special: Auto-calculate line totals (qty * unit_price - discount). Real-time calculation of grand total. Line item quantity selector dengan available stock display

Flow: Click 'New SO' -> Form (Tabs) -> Add Line Items -> Specify Customer & Dates -> Submit -> Validate -> Create Order -> Show SO Detail

2.3 Daftar Invoice

File Path: sales/invoices_list.html

UI/UX: Tabel: Invoice No, Customer, Invoice Date, Due Date, Amount, Status (Draft, Sent, Paid, Partially Paid, Overdue, Cancelled). Status indicator colors. Filter: Status, Date Range, Overdue. Search: Invoice No atau Customer. Action: View, Edit, Send, Print, Record Payment

Backend: GET: Fetch invoices + calculate payment status (paid_amount vs invoice_total). Calculate overdue days. Prefetch customer & line items

Database: SalesInvoices (main), InvoiceLines (FK), Payments (FK for paid_amount aggregate). LumraConfigCustomers (FK)

Calculation: paid_amount = SUM(amount) FROM Payments WHERE invoice_id = this. status = 'Paid' if paid_amount >= invoice_total, 'Overdue' if due_date < today AND paid_amount < invoice_total

Flow: Sales -> Invoices -> List -> Filter by Status -> Click Invoice -> View / Print / Record Payment / Send

2.4 Detail Invoice & Actions

File Path: sales/invoice_detail.html

UI/UX: Display: Invoice header (No, Date, Customer, Due Date), line items table (Product, Qty, Unit Price, Amount), payment section (Amount due, Payment history tabel). Buttons: Print, Email, Record Payment, Create Credit Note, Cancel Invoice

Backend: GET: Fetch invoice detail, line items, payments. Calculate: amount_due = total - sum(paid amounts)

POST (Record Payment): Create Payments record (amount, date, method). Update invoice status

POST (Create Credit Note): Create CreditNotes record, link ke invoice. Reduce AR.

Database: SalesInvoices (read), InvoiceLines (read), Payments (read/write), CreditNotes (write), GeneralLedger (write for accounting)

Print: Use WeasyPrint untuk render invoice PDF dengan invoice template (header, items, totals, terms)

Flow: Invoice Detail -> Click 'Record Payment' -> Modal (Amount, Method, Date) -> Submit -> Create Payment Record -> Update Status -> Show Success

3. INVENTORY ADVANCED

3.1 Stock Opname Workflow - Overview

Stock Opname adalah proses fisik menghitung stok untuk memastikan database akurat. Workflow terdiri dari 5 halaman utama: Pilih Lokasi -> Input Hitungan -> Daftar Approval -> Detail Approval -> Session History

Database Models:

- StockOpnameSessions: header (session_id, location_id, status, start_date, end_date, created_by)
- StockOpnameLines: detail (session_id, product_id, system_qty, counted_qty, difference, variance_pct, status)
- StockOpnameApprovals: approval flow (session_id, approver_id, status, notes, approved_at)

Status Flow: DRAFT -> IN_PROGRESS -> COUNTING_COMPLETE -> PENDING_APPROVAL -> APPROVED -> POSTED

3.2 Stock Opname - Select Location

File Path: inventory/opname_select_location.html

UI/UX: Lokasi list dengan status indicator (In Progress, Completed, Draft). Tabel: Location Name, Last Opname Date, Status, Last Counted By. Button: 'Start New Session', 'Continue Existing'

Backend: GET: Fetch all locations + last opname session per location. Show only locations where user has permission

Database: LumraConfigLocations, StockOpnameSessions (for last session info)

Flow: Inventory -> Stock Opname -> Select Location -> Click 'Start New' -> Create new session -> Go to Input Hitungan

3.3 Stock Opname - Input Hitungan

File Path: inventory/opname_input_counting.html

UI/UX: Tabel dengan columns: SKU, Product Name, System Qty, Counted Qty (input field), Difference, Variance %. Products bisa filtered/searched. Counted Qty field auto-calculate difference & variance. Progress bar showing completion %

Backend: GET: Fetch products dalam location, system_qty dari InventoryTransaction. Create empty StockOpnameLines

- AJAX POST untuk setiap counted_qty input: Calculate difference, validate (not negative)
- Prevent counting qty > (system_qty * 150%) to catch input errors

Database: StockOpnameLines (write: counted_qty, difference, variance_pct). InventoryTransaction (read: system_qty)

Special: Drag-drop reorder items jika perlu. Mark item 'Done' saat selesai count. Session auto-save every 2 minutes. Variance report untuk items dengan >10% difference

Flow: Input counting -> Auto-calculate variance -> Mark items as done -> Click 'Complete Counting' -> Go to Approval

3.4 Stock Opname - Approval List & Detail

File Path: inventory/opname_approval_list.html, opname_approval_detail.html

Approval List UI: Tabel: Session ID, Location, Counted By, Count Date, Status (Pending, Approved, Rejected), Items with Variance. Filter: Status, Location. Action: View Detail, Approve, Reject

Detail Page UI: Summary: Location, Date, Counted By. Tab: Items (tabel dengan SKU, Product, System Qty, Counted Qty, Variance). Tab: Variance Report (items >10% diff dengan red highlight). Approval section: Approver notes, Approve/Reject buttons

Backend - Approve: Validation: At least 1 item counted. Calculate & verify variance < threshold (default 5%). Create InventoryTransaction untuk adjust stok (counted_qty - system_qty). Update session status to POSTED. Audit log

Backend - Reject: Reset session to IN_PROGRESS. Add note ke approvals table. Notify counter untuk re-count

Database: StockOpnameSessions (write: status), StockOpnameApprovals (write), InventoryTransaction (write for adjustments)

Flow: Approval List -> Click session -> Review items & variance -> Approve -> Create adjustment transactions -> Update stok -> Success notification

3.5 Stock Opname - Session History

File Path: inventory/opname_session_history.html

UI/UX: Timeline atau tabel history semua opname sessions dengan: Date, Location, Item Count, Variance Count, Approver, Status. Link ke detailed report untuk setiap session

Backend: GET: Fetch all completed StockOpnameSessions ordered by date DESC. Show summary metrics

Database: StockOpnameSessions (read)

Export: CSV export untuk audit trail

Flow: Inventory -> Stock Opname -> Session History -> Click session -> View full opname report

4. STOCK TRANSFER BETWEEN LOCATIONS

4.1 Stock Transfer Form & Workflow

File Path: inventory/stock_transfer_form.html

UI/UX: Form dengan: From Location (dropdown), To Location (dropdown), Items (tabel: Product, Qty, Status). Add items button. Submit creates transfer order

Backend: GET (form): Load locations dropdown. POST: Validate from != to location, items qty > 0 & <= available qty in from location. Create StockTransfers record + StockTransferLines. Create InventoryTransaction 2x: OUTGOING (from location), INCOMING (to location)

Database: StockTransfers (write), StockTransferLines (write), InventoryTransaction (write: 2 records per transfer)

Flow: Inventory -> Stock Transfer -> Select Locations -> Add Items -> Submit -> Create Transfer -> Stock auto-updated

Special: Real-time balance update. Prevent negative stock. Optional approval step jika amount > threshold

5. LOYALTY & CUSTOMER ENGAGEMENT

5.1 Loyalty Dashboard

File Path: sales/loyalty_dashboard.html

UI/UX: KPI cards: Total Members, Active Members, Total Stamps Issued, Total Rewards Redeemed. Tier distribution chart. Reward popularity. Recent redemptions list

Backend: GET: Aggregate dari LoyaltyStampCards, LoyaltyTiers, LoyaltyRedemptions

Database: LoyaltyStampCards (count members), LoyaltyTiers (tier distribution), LoyaltyRedemptions (count rewards)

Flow: Sales -> Loyalty -> Dashboard -> View metrics

5.2 Stamp Card Management

File Path: sales/loyalty_stamp_cards_list.html

UI/UX: Tabel: Card Number, Customer, Tier, Current Stamps, Lifetime Stamps, Last Visit, Status. Filter: Tier, Status (Active/Inactive). Search: Card No atau Customer. Action: View Card, Manage Stamps, Deactivate

Backend: GET: Fetch cards + prefetch customer, tier. Calculate stamps progress to next reward

Database: LoyaltyStampCards (main), LoyaltyTiers (FK), LumraConfigCustomers (FK)

Manual Stamps: POST endpoint untuk manually add/deduct stamps (admin only) dengan reason

Flow: Loyalty -> Stamp Cards -> List -> Click Card -> View Customer Details / Manual Adjust / View Transactions

5.3 Redemption History & Rewards

File Path: sales/loyalty_redemptions_list.html, loyalty_rewards_list.html

Redemptions List UI: Tabel: Card No, Customer, Reward, Stamps Used, Date, Status. Filter: Status, Date Range. Action: View, Cancel Redemption

Rewards List UI: Tabel: Code, Name, Type (Free Item, Discount, Merchandise), Required Stamps, Valid Days, Redeemed Count, Status. Action: View, Edit, Activate/Deactivate

Backend: GET redemptions: Fetch dari LoyaltyRedemptions + prefetch card, reward, order. GET rewards: Fetch rewards + count redeemed per reward

Database: LoyaltyRedemptions (read/write), LoyaltyRewards (read/write), LoyaltyStampCards (read)

Flow: Loyalty -> Redemptions -> View history | Loyalty -> Rewards -> Manage reward catalog

6. LAMPIRAN & BEST PRACTICES

6.1 POS Workflow - Detailed Flow Diagram

START -> Dashboard (stats) -> New Transaction
-> Product Selection (search/category) -> Add to Cart (AJAX)
-> Review Cart (edit qty, remove items) -> Discount Application (optional)
-> Customer Selection (optional) -> Payment Selection (Cash/Card/Other)
-> Create Order (save to DB) -> InventoryTransaction (deduct stok)
-> LoyaltyStampTransactions (earn stamps jika customer present) -> Print Receipt
-> Update Dashboard Stats -> Transaction Complete

ERROR HANDLING:

- Stock sold out during checkout -> Show warning, auto-remove from cart
- Product price changed -> Alert user, ask confirmation
- Discount invalid -> Show error, block payment
- Payment failed -> Rollback transaction, show error message
- Network disconnect -> Save to localStorage, retry on reconnect

6.2 Stock Opname Approval Workflow

START -> Select Location
-> Create Session (DRAFT) -> Input Counting (status: IN_PROGRESS)
-> Calculate Variance -> Mark Complete (COUNTING_COMPLETE)
-> Send to Approver (PENDING_APPROVAL) -> Review Variance Report
-> Approver Decision: APPROVE or REJECT

IF APPROVE:

-> Validate variance < threshold (5%)
-> Create InventoryAdjustment (counted_qty - system_qty)
-> Create InventoryTransaction records
-> Update ProductItems.available_qty
-> Mark Session POSTED

ELSE (REJECT):

-> Reset to IN_PROGRESS
-> Notify counter untuk re-count

END

6.3 Performance Optimization Tips

- POS Terminal: Use select_for_update() saat update inventory untuk prevent race condition
- Cache: Cache product list & categories 10 min. Invalidate saat product update
- Query: Always prefetch_related & select_related untuk avoid N+1 queries
- Cart: Store di session/localStorage untuk fast access tanpa DB hit every time
- Pagination: Max 50 items per page untuk list views. Use cursor pagination untuk large datasets
- Stok calculation: Don't recalculate setiap kali. Cache & update incrementally
- Loyalty: Cache member tier status 5 min untuk fast lookup saat add to cart
- Reporting: Use database views atau pre-calculated tables (materialized views) untuk complex reports

LUMRA ERP

Page Architecture Blueprint

BAGIAN 3: PRODUCTION, ACCOUNTING & REPORTS

Disusun: 05 May 2026

DAFTAR ISI - BAGIAN 3

1. PRODUCTION MANAGEMENT

- Production Order List
- Create Production Order
- Production Order Detail & Execution
- Material Consumption Tracking
- Finished Goods Receipt

2. ACCOUNTING & GENERAL LEDGER

- Chart of Accounts
- General Ledger Entries
- Journal Entry Form
- Account Reconciliation

3. FINANCIAL REPORTS

- Balance Sheet
- Profit & Loss Statement
- Cash Flow Statement
- Trial Balance

4. OPERATIONAL REPORTS

- Sales Report (Daily/Monthly)
- Inventory Report
- Purchase Report
- Employee Performance

5. CUSTOM REPORT BUILDER

6. DASHBOARD & ANALYTICS

7. INTEGRATION & API BEST PRACTICES

8. LAMPIRAN FINAL

1. PRODUCTION MANAGEMENT

1.1 Production Order List

File Path: production/production_orders_list.html

UI/UX: Tabel: PO No, Output Product, Qty, Status (Draft, Confirmed, In Progress, Completed, Cancelled), Scheduled Date, Actual Date, Assigned To. Filter: Status, Date Range, Product. Search: PO No atau Product. Action: View, Edit (if Draft), Execute, Complete, Cancel

Backend: GET: Fetch ProductionOrders + prefetch product, user. Calculate completion % based on FinishedGoodsReceipts

Database: ProductionOrders (main), ProductionOrderLines (FK), FinishedGoodsReceipts (for qty completed tracking)

Flow: Production -> Orders -> List -> Click PO -> View / Edit / Execute / Complete

Status Colors: Draft=Gray, Confirmed=Blue, In Progress=Orange, Completed=Green, Cancelled=Red

1.2 Create Production Order

File Path: production/production_order_form.html

UI/UX: Tabs: Order (PO No auto-generate, Output Product dropdown, Target Qty, Scheduled Date, Notes), Ingredients (Tabel dari Recipe: Product, Required Qty, Unit, Available Qty with warning if low). Assign: Assign to operator

Backend: GET (create): Output product dropdown, recipe list. GET (edit): Pre-fill PO data. POST: Validate recipe exists for product, qty > 0. Create ProductionOrders + fetch recipe ingredients -> create ProductionOrderLines

Auto-Calculate: When output product selected, auto-load recipe from Recipes table. Show required materials

Database: ProductionOrders (write), ProductionOrderLines (write from recipe), Recipes (read for ingredients)

Flow: Click 'New PO' -> Select Output Product (auto-load recipe) -> Enter Qty & Date -> Auto-populate ingredients -> Assign to operator -> Save -> Show PO Detail

1.3 Production Order Detail & Execution

File Path: production/production_order_detail.html

UI/UX: Display: PO header (No, Product, Target Qty, Status, Assigned To). Tab 1 (Materials): Tabel ingredients dengan Required Qty, Used Qty (updated from MaterialConsumption), Balance, Status (On Hand, Shortage). Tab 2 (Progress): Steps (Start Production, Consume Materials, Finish & Receipt). Tab 3 (History): Timeline of all actions

Actions Buttons: 'Start Production' (change status to In Progress), 'Finish & Receipt' (record finished goods)

Material Consumption: Operator can manually log consumed materials via Material Consumption Tracking form

Finished Goods Receipt: Click 'Finish & Receipt' -> Enter received qty -> Create FinishedGoodsReceipts record -> Increase inventory -> Status = Completed

Database: ProductionOrders (read/write status), ProductionOrderLines (read), MaterialConsumption (read), FinishedGoodsReceipts (write)

Flow: PO Detail -> Start -> Log Material Consumption -> Click Finish -> Record Receipt Qty -> Complete PO -> Update Inventory

1.4 Material Consumption Tracking

File Path: production/material_consumption_form.html

UI/UX: Modal atau separate page. Show PO & materials. Tabel: Material, Required Qty, Previously Consumed, Qty to Consume (input field), Balance. Submit button creates material consumption record

Backend: GET: Show PO materials + sum previous consumption. POST: Create MaterialConsumption record (po_id, material_id, qty_consumed, timestamp). Validate: consumed qty <= required qty. Update balance display in real-time (AJAX)

Database: MaterialConsumption (write), ProductionOrderLines (read: required_qty)

Error Handling: If consumed qty > required qty -> show warning but allow (for overage tracking)

Flow: PO Detail -> Tab 'Materials' -> Click 'Log Consumption' -> Form -> Enter qty -> Submit -> Update material balance in real-time

1.5 Finished Goods Receipt

File Path: production/finished_goods_receipt_form.html

UI/UX: Modal. Show PO info + output product. Input: Received Qty, Received Date, Quality Status (Pass/Fail/Partial). Notes field for defects or issues

Backend: POST: Validate received_qty > 0 and <= target_qty (or allow overage?). Create FinishedGoodsReceipts record. Create InventoryTransaction to increase inventory. If received_qty < target_qty -> create variance record for investigation. Update ProductionOrders.status to COMPLETED

Database: FinishedGoodsReceipts (write), InventoryTransaction (write), ProductionOrders (update status)

Flow: PO Detail -> Finish & Receipt -> Modal -> Enter received qty & quality -> Submit -> Create receipt & update inventory -> PO Complete

2. ACCOUNTING & GENERAL LEDGER

2.1 Chart of Accounts

File Path: accounting/chart_of_accounts.html

UI/UX: Tree view hierarchical accounts (Asset, Liability, Equity, Income, Expense). Tabel columns: Account Code, Account Name, Type (Debit/Credit), Balance, Status. Action: View, Edit, Activate/Deactivate, View Ledger

Backend: GET: Fetch ChartOfAccounts dengan parent-child structure. Calculate balance per account dari GeneralLedger (SUM debit - credit)

Database: ChartOfAccounts (main with parent_id FK), GeneralLedger (for balance calculation)

Flow: Accounting -> Chart of Accounts -> Tree View -> Click account -> View Ledger / Edit / Deactivate

2.2 General Ledger Entries

File Path: accounting/general_ledger_list.html

UI/UX: Tabel: Date, Journal (manual entry / auto-generated from sales/purchase), Description, Debit Amount, Credit Amount, Reference No, Post Status (Draft/Posted). Filter: Date Range, Journal Type, Account. Search: Description atau Reference. Action: View, Edit (if Draft), Post, Reverse

Backend: GET: Fetch GeneralLedger entries. Show only posted entries (unless user is admin). Pagination with 50 entries per page

Database: GeneralLedger (main), ChartOfAccounts (FK)

Flow: Accounting -> General Ledger -> List -> Filter -> View Entry / Post / Reverse

2.3 Journal Entry Form (Manual)

File Path: accounting/journal_entry_form.html

UI/UX: Form: Entry Date, Journal Name (dropdown), Description, Line Items (tabel: Account dropdown, Debit, Credit, Description). Add/remove line buttons. Auto-validate: Total Debit = Total Credit

Backend: GET: Load chart of accounts for dropdown. POST: Validate total debit = total credit, all accounts exist, at least 2 lines. Create JournalEntries record + GeneralLedger entries (auto-generated)

Database: JournalEntries (write), GeneralLedger (write), ChartOfAccounts (read for dropdown)

Special: Status initially Draft. Admin posts entries for final approval

Flow: Accounting -> Journal Entries -> New -> Form -> Add lines -> Submit -> Validate balance -> Create entry (Draft) -> Post (admin only)

2.4 Account Reconciliation

File Path: accounting/account_reconciliation.html

UI/UX: Form: Select Account, Start Date, End Date, Bank Statement Balance (manual input), System Balance (auto-calculated). Reconciling items table: Outstanding Checks, Deposits in Transit, Errors. Difference amount with alert if not zero

Backend: GET: Select account -> show system balance (from GeneralLedger). Calculate: System Balance + Reconciling Items = Bank Statement Balance

Database: GeneralLedger (for system balance), Reconciliation (store reconciliation record)

Flow: Accounting -> Reconciliation -> Select Account & Date -> Compare with bank statement -> Add reconciling items -> Validate -> Post reconciliation

3. FINANCIAL REPORTS

3.1 Balance Sheet

File Path: reports/balance_sheet.html

UI/UX: Form: Report Date (datepicker). Display: 3 main sections (Assets, Liabilities, Equity) dengan sub-accounts indented. Tabel columns: Account, Amount. Totals & subtotals bolded. Export button (PDF, Excel)

Backend: GET: Report Date parameter -> fetch all accounts type Asset, Liability, Equity -> sum balances per account dari GeneralLedger. Calculate totals. Validate: Assets = Liabilities + Equity

Database: GeneralLedger (aggregate), ChartOfAccounts (structure)

PDF Export: Use WeasyPrint untuk render balance sheet in PDF format dengan company header

Flow: Reports -> Balance Sheet -> Select Date -> View Report -> Export PDF/Excel

3.2 Profit & Loss Statement

File Path: reports/profit_loss_statement.html

UI/UX: Form: Date Range (from-to datepickers). Display: Income - Expenses = Net Profit. Sections: Revenue (all income accounts), Expenses (all expense accounts), Operating Income, Other Income/Expense

Backend: GET: Date range -> fetch Income & Expense accounts -> sum amounts per account. Calculate: Total Revenue - Total Expenses = Net Profit

Database: GeneralLedger (aggregate by date range)

Flow: Reports -> P&L; -> Select Date Range -> View -> Export

3.3 Cash Flow Statement

File Path: reports/cash_flow_statement.html

UI/UX: Form: Date Range. Display: 3 sections (Operating Activities, Investing Activities, Financing Activities) + Beginning & Ending Cash Balance

Backend: GET: Complex calculation from GeneralLedger. Require manual mapping of accounts to activity categories (setup in admin)

Database: GeneralLedger, AccountActivityMapping (admin config)

Note: This requires predefined account mappings to activity types. Setup in admin before report can work

Flow: Reports -> Cash Flow -> Select Date Range -> View (requires account mapping setup) -> Export

3.4 Trial Balance

File Path: reports/trial_balance.html

UI/UX: Form: Report Date. Tabel: Account Code, Account Name, Debit Balance, Credit Balance. Totals row showing total debit & credit

Backend: GET: Report Date -> fetch all active accounts -> calculate debit & credit per account. Validate: Total Debit = Total Credit

Database: GeneralLedger (aggregate), ChartOfAccounts (list)

Alert: If total debit != total credit -> show error. Likely indicates posting errors

Flow: Reports -> Trial Balance -> Select Date -> View -> Export

4. OPERATIONAL REPORTS

4.1 Sales Report (Daily/Monthly)

File Path: reports/sales_report.html

UI/UX: Form: Date Range, Group By (Day/Week/Month), Filter by Product Category, Sales Channel (POS/SO). Display: Tabel atau chart. Columns: Date, Product Category, Qty Sold, Revenue, Avg Price. Summary: Total Revenue, Total Qty, Avg Transaction Value

Backend: GET: Date range + filters -> aggregate dari OrderLines. Group by date & category. Calculate: SUM(qty), SUM(revenue), AVG(price)

Database: Orders (filter by date), OrderLines (details), ProductItems (for category)

Chart: Line chart for revenue trend. Bar chart for product category comparison

Flow: Reports -> Sales -> Select Date Range & Filters -> View Report -> Export CSV/PDF

4.2 Inventory Report

File Path: reports/inventory_report.html

UI/UX: Form: Report Date, Location filter, Category filter. Display: Tabel: SKU, Product, Current Qty, Min Level, Max Level, Reorder Qty, Turnover Rate, Stock Value (qty * cost). Highlight: Low stock (yellow), Out of stock (red), Slow movers (blue)

Backend: GET: All products -> calculate current qty (from InventoryTransaction), turnover (sold last 30 days / avg qty). Show slow movers & overstock

Database: ProductItems, InventoryTransaction (for qty & turnover), OrderLines (for sold qty)

Flow: Reports -> Inventory -> Select filters -> View Report -> Identify issues -> Export

4.3 Purchase Report

File Path: reports/purchase_report.html

UI/UX: Form: Date Range, Vendor filter. Display: Tabel: PO No, Vendor, Product, Qty, Total Amount, Status. Summary: Total Purchase Amount, Total Qty, Top Vendors

Backend: GET: Date range -> aggregate dari PurchaseOrders & PurchaseOrderLines

Database: PurchaseOrders, PurchaseOrderLines, LumraConfigVendors

Flow: Reports -> Purchase -> Select filters -> View Report -> Export

5. CUSTOM REPORT BUILDER

5.1 Query Builder Interface

File Path: reports/custom_report_builder.html

UI/UX: Drag-drop interface untuk build custom report. Step 1: Select Table (Orders, Products, etc). Step 2: Select Columns to display. Step 3: Add Filters (conditions). Step 4: Grouping & Aggregation (SUM, COUNT, AVG). Step 5: Preview & Export

Backend: Build Django QuerySet dynamically based on user selections. Validate selections for security (prevent SQL injection)

Database: Any model defined in project can be queried

Special: Save custom reports as template for future use. Schedule report generation (daily/weekly email)

Flow: Reports -> Custom Report -> Select Table -> Columns -> Filters -> Preview -> Save Template -> Export

6. DASHBOARD & ANALYTICS

6.1 Main Executive Dashboard

File Path: dashboard/main_dashboard.html

UI/UX: KPI cards: Today Sales, Monthly Revenue, Active Users, Inventory Value, Pending POs. Charts: Revenue trend (line), Sales by category (pie), Top products (bar). Quick stats: Today transactions, Yesterday comparison, Month-to-date

Backend: Aggregate dari Orders, ProductItems, Users. Use cache untuk performa (5 min TTL). Prefetch metrics saat user login

Database: Orders (sales metrics), ProductItems (inventory), Users (active count)

Real-time: WebSocket untuk live update POS sales (optional, nice-to-have)

Flow: User login -> Dashboard loads -> Show KPIs & trends -> Drill-down to detailed reports

6.2 Department-Specific Dashboards

Sales Manager Dashboard: Top customers, sales rep performance, revenue vs target, pending orders

Inventory Manager Dashboard: Low stock alert, overstock, slow movers, stock opname status

Accounting Manager Dashboard: Outstanding AR, AP aging, trial balance status, GL posting status

Production Manager Dashboard: Production orders status, material consumption, finished goods received, yield variance

Implementation: Each dashboard GET from respective models, filter by department/user role

7. INTEGRATION & API BEST PRACTICES

7.1 REST API Standard Endpoints

- GET /api/products/ -> List products with pagination, filtering
- GET /api/products/[id]/ -> Product detail
- POST /api/products/ -> Create product
- PUT /api/products/[id]/ -> Update product
- DELETE /api/products/[id]/ -> Soft delete product
- GET /api/orders/ -> List orders
- POST /api/orders/ -> Create order (POS, SO)
- GET /api/orders/[id]/detail/ -> Order detail with lines
- POST /api/inventory/stock-adjustment/ -> Create stock adjustment
- GET /api/reports/sales-report/?date_from=X&date_to=Y -> Sales report (parametric)
- GET /api/dashboard/kpis/ -> Dashboard metrics

All endpoints require JWT token authentication (Authorization: Bearer TOKEN)

Throttling: 1000 requests/hour per user

Response format: {"success": bool, "data": {}, "message": string, "errors": {}}

7.2 Serializers & Validation

Use DRF ModelSerializer untuk auto-generate validators dari model fields. Custom validators untuk business logic (e.g., code uniqueness, stock availability). Nested serializers untuk related objects (Order + OrderLines in single endpoint). Pagination: LimitOffsetPagination dengan max_limit=100. Filtering: DjangoFilterBackend + SearchFilter untuk full-text search

7.3 Async Tasks (Celery)

Heavy operations di background tasks:

- Generate report PDF (> 100 pages) -> Task, email link
- Bulk import products (CSV) -> Task, notify on completion
- Send invoice email -> Task queue
- Calculate loyalty stamps -> Task (scheduled, hourly)
- Stock level alerts -> Periodic task (every hour)

Use Celery Beat untuk scheduled tasks. Store task results di Redis untuk quick retrieval

8. LAMPIRAN FINAL

8.1 Database Indexing Strategy

Create indexes on frequently queried fields:

- ProductItems: code (unique), name (for search)
- Orders: customer_id, order_date, status
- InventoryTransaction: product_id, location_id, transaction_date
- GeneralLedger: account_id, posting_date, journal_id
- StockOpnameSessions: location_id, created_at, status

Use compound indexes untuk common filter combinations

Monitor slow queries: Enable Django Debug Toolbar in dev, use Silk profiler in production

8.2 Security Best Practices

- CSRF Protection: Always include {% csrf_token %} in forms
- Permission Checks: @permission_required decorator on every sensitive view
- Rate Limiting: Throttle API endpoints to prevent abuse
- Input Validation: Sanitize & validate ALL user inputs (forms & API)
- SQL Injection Prevention: Use Django ORM QuerySet, never raw SQL unless absolutely necessary
- Soft Delete: Never show deleted data unless explicitly requested by admin
- Audit Logging: Log all create/update/delete operations with user & timestamp
- Two-Factor Auth: Implement 2FA for admin users
- HTTPS Only: Force HTTPS in production (DEBUG=False, SECURE_SSL_REDIRECT=True)

8.3 Testing Strategy

Unit Tests: Test individual models & view functions

Integration Tests: Test workflows (e.g., POS full transaction flow)

API Tests: Test all API endpoints with valid & invalid inputs

Load Tests: Test high-concurrency scenarios (many POS terminals at same time)

Fixtures: Create test data for repeatable test runs

Coverage: Aim for >80% code coverage on critical modules

CI/CD: Run tests automatically on every commit (GitHub Actions / Jenkins)

Example: @pytest.mark.django_db class TestPOSCheckout: def test_create_order_with_valid_items(): ...

8.4 Deployment Checklist

Pre-Deployment:

- All tests passing (coverage > 80%)
- Code review completed
- Database migrations reviewed
- Static files optimized (minified CSS/JS)
- Secrets (API keys, DB password) in environment variables
- Logging configured (Sentry for error tracking)

Post-Deployment:

- Health check monitoring (uptime, response time)
- Error tracking active (Sentry alerts)
- Database backups scheduled
- Cache warmed up
- Performance monitoring (slow queries, N+1 queries)
- User support ready for issues

8.5 Project Statistics

Total Halaman (Pages): ~60 halaman web + dashboard + reports

Total Models: ~50 Django models (dari rekap)

Total Views: ~80 view functions (list, form, detail, API)

Total API Endpoints: ~30 REST endpoints

Total Templates: ~100+ HTML templates

Estimated LOC (Lines of Code): ~20,000 LOC (Python + HTML + JS)

Database Tables: ~200+ tables

Average Load Time: < 2 seconds per page (with caching)

Concurrent Users Supported: 100-500+ with load balancing

8.6 Next Steps for Implementation

1. Start dengan Master Data & Settings (Bagian 1) - est. 2-3 weeks
2. Implement POS & Sales (Bagian 2) - est. 3-4 weeks
3. Add Production & Accounting (Bagian 3) - est. 3-4 weeks
4. Create Reports & Analytics - est. 2-3 weeks
5. Security hardening & penetration testing - est. 1-2 weeks
6. Performance tuning & load testing - est. 1-2 weeks
7. User training & documentation - est. 2 weeks
8. Go-live & support - est. ongoing

Total Estimated Timeline: 16-21 weeks (~4-5 months) dengan team 3-5 developers